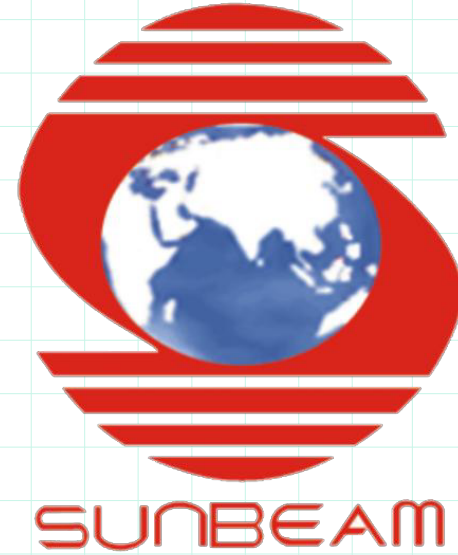




GIT

Trainer: Nilesh Ghule

Section 0: GIT Fundamentals

Commands: git add, git commit, git push, git pull, git clone

Learning Objectives:

- Understand VCS & GIT
- GIT terminologies & architecture
- Navigate between commits and branches
- Understand different reset modes
- Restore files to previous states

Version Control Systems (VCS)

VCS tracks changes to files over time

Why VCS is Important:

- Track history of changes
- Collaborate with team members
- Backup and recovery
- Branching for parallel development
- Blame/annotation for debugging

Examples: Git, SVN, Mercurial, Perforce

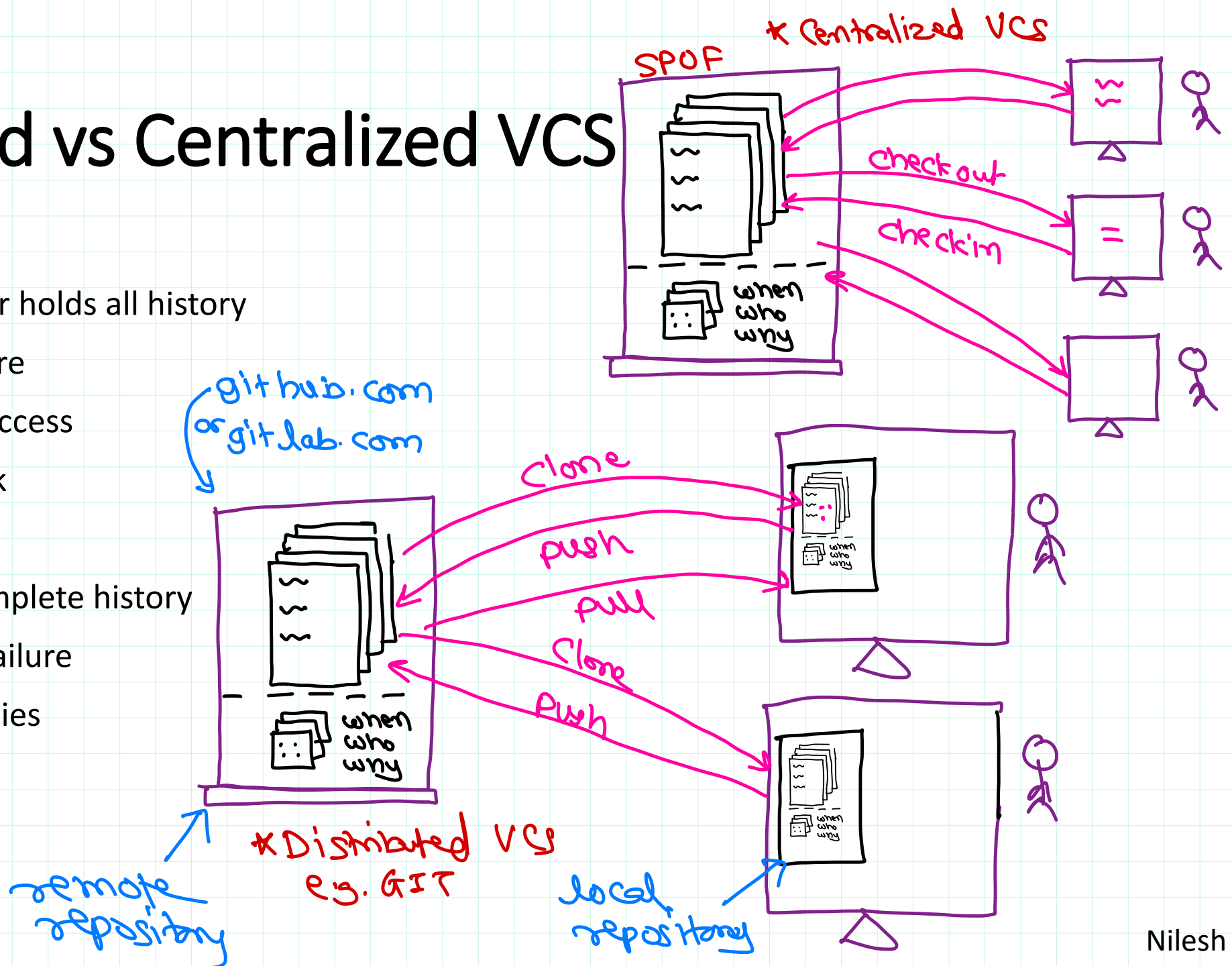
Distributed vs Centralized VCS

Centralized VCS:

- Single central server holds all history
- Single point of failure
- Requires network access
- Limited offline work

Distributed VCS:

- Every clone has complete history
- No single point of failure
- Full offline capabilities
- Flexible workflows





Git Repository Basics

- Repository: Container for your project and its history
- Commit: Snapshot of your project at a point in time
- Branch: Movable pointer to a specific commit
- HEAD: Pointer to current branch/commit
- Working Directory: Files you're currently working on

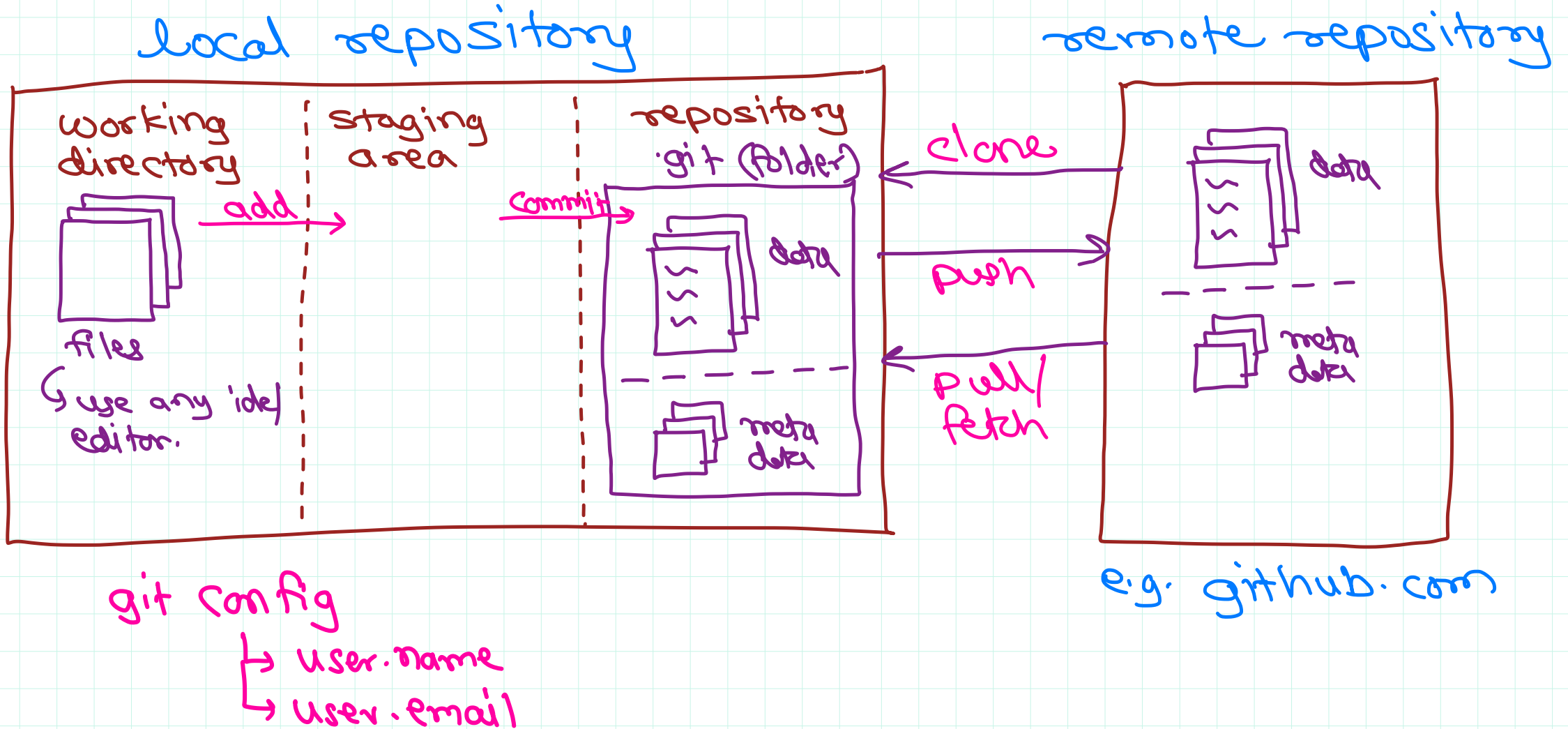
GIT → Linus Torvalds.

2005 → CVS ×

2005/06 → Distributed VCS
→ GIT

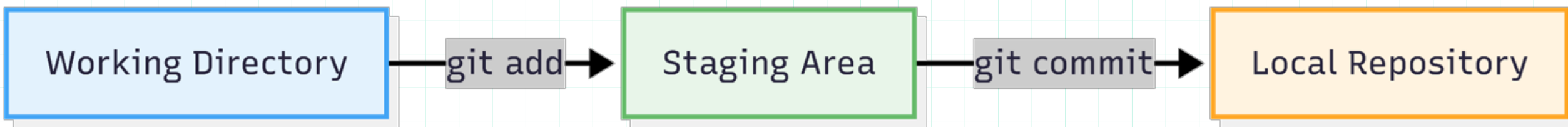
↓
opensource/
Free

GIT Architecture



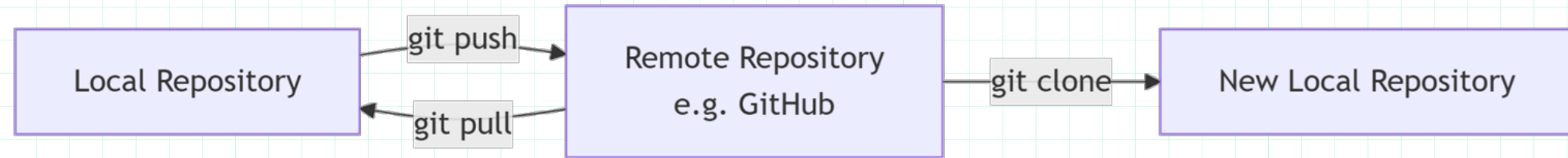
GIT Architecture - Local Repository Structure

- Working Directory → Your actual files (Modified files)
- Staging Area (Index) → Files ready for commit (Staged files)
- .git Directory → Git's database (Committed files)



Remote Repository Concepts

- Remote: Version of your project hosted elsewhere
- Origin: Default name for primary remote repository
- GitHub: Popular hosting service for Git repositories



Common Workflow Review

- 1. git status - Check current state
- 2. git add <file> - Stage changes
- 3. git commit -m 'message' - Create snapshot
- 4. git log - View history
- 5. git pull - Get new changes from remote repo
- 6. git push - Push local repo changes to remote repo

Section 1: Status, Compare, Reset, Stash

Commands: git status, git diff, git checkout, git reset

Learning Objectives:

- Understand Git status codes
- Compare changes between versions
- Navigate between commits and branches
- Understand different reset modes
- Restore files to previous states

git status -s Status Codes

Shows the status of files using a concise, two-letter code format.

1st char: status of the staging area, while 2nd char: status of the working dir

??	An untracked file, not yet added to the repository.
A	A new file that has been added to the staging area.
M	A file has been modified in the working directory.
M	A file has been modified and staged, but not yet committed.
D	A file has been deleted from the working directory.
D	A file has been deleted from the working directory and the deletion has been staged.
U	A file has been updated but is currently unmerged.
UU	A file is unmerged, meaning there are merge conflicts.

git diff - Compare Changes

Purpose: Compare changes between commits, branches, or files

Syntax: `git diff [options] [<commit>] [--] [<path>...]`

Common Uses:

- `git diff` - Show unstaged changes
- `git diff --staged` - Show staged changes
- `git diff HEAD~1` - Compare with previous commit
- `git diff main feature` - Compare two branches

git checkout - Navigate & Restore

Purpose: Switch branches or restore files

Syntax: `git checkout [<branch>]` OR `git checkout [<commit>] -- [<file>]`

- `git checkout main` - Switch to main branch
- `git checkout -b feature` - Create and switch to new branch
- `git checkout -- file.txt` - Restore file from last commit
- `git checkout abc123 -- file.txt` - Restore file from specific commit

⚠ ⚠ Checkout can discard unsaved changes

git reset - Reset Repository State

Purpose: Reset HEAD, staging area, and working directory

Syntax: `git reset [--soft | --mixed | --hard] [<commit>]`

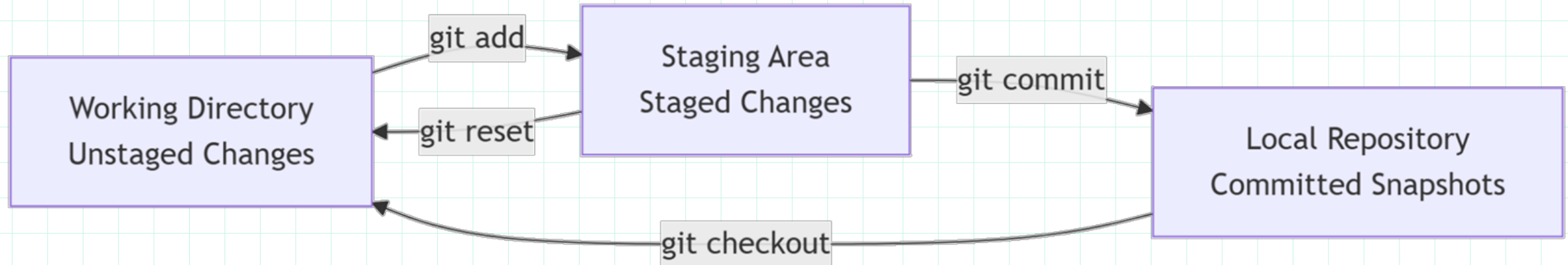
Modes:

- `--soft`: Only move HEAD pointer (✓ Reset HEAD, ✗ Staging, ✗ Working)
- `--mixed` (default): Reset HEAD and staging (✓ Reset HEAD, ✓ Staging, ✗ Working) - *changes unstaged (in working directory)*
- `--hard`: Reset everything - DANGEROUS! (✓ Reset HEAD, ✓ Staging, ✓ Working)

changes remain staged.

⚠ ⚠ Hard reset permanently discards changes!

git add, commit, reset, & checkout



git stash

Purpose: Temporarily saves uncommitted changes in a clean state so you can work on something else.

Syntax: `git stash [ list | pop [ stash@{n} ] | show -p | apply | drop stash@{n} ]`

stash: Saves uncommitted changes onto a temp stack and clean cwd

list: List of all the stashes created, with the latest at the top as `stash@{0}`.

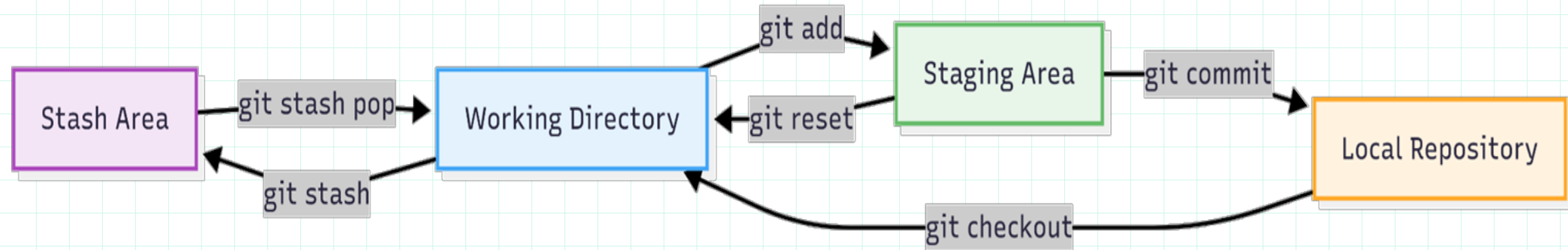
pop: Reapplies the most recent stash to your working directory and then deletes it.

show: Shows stashed contents

apply: Reapplies a stash to your working directory but keeps it in the stash stack.

`drop stash@{n}`: Deletes a specific stash from the stack.

git add, commit, reset, checkout & stash



Section 2: Branching & Merging

Commands: git branch, git checkout, git merge

Learning Objectives:

- Create and manage branches
- Switch between branches
- Understand merge strategies
- Handle merge conflicts

branching enables
↳ parallel development
↓
can develop multiple
features parallelly
(by one feature dev).
with min clash.

branch → sequence of commits

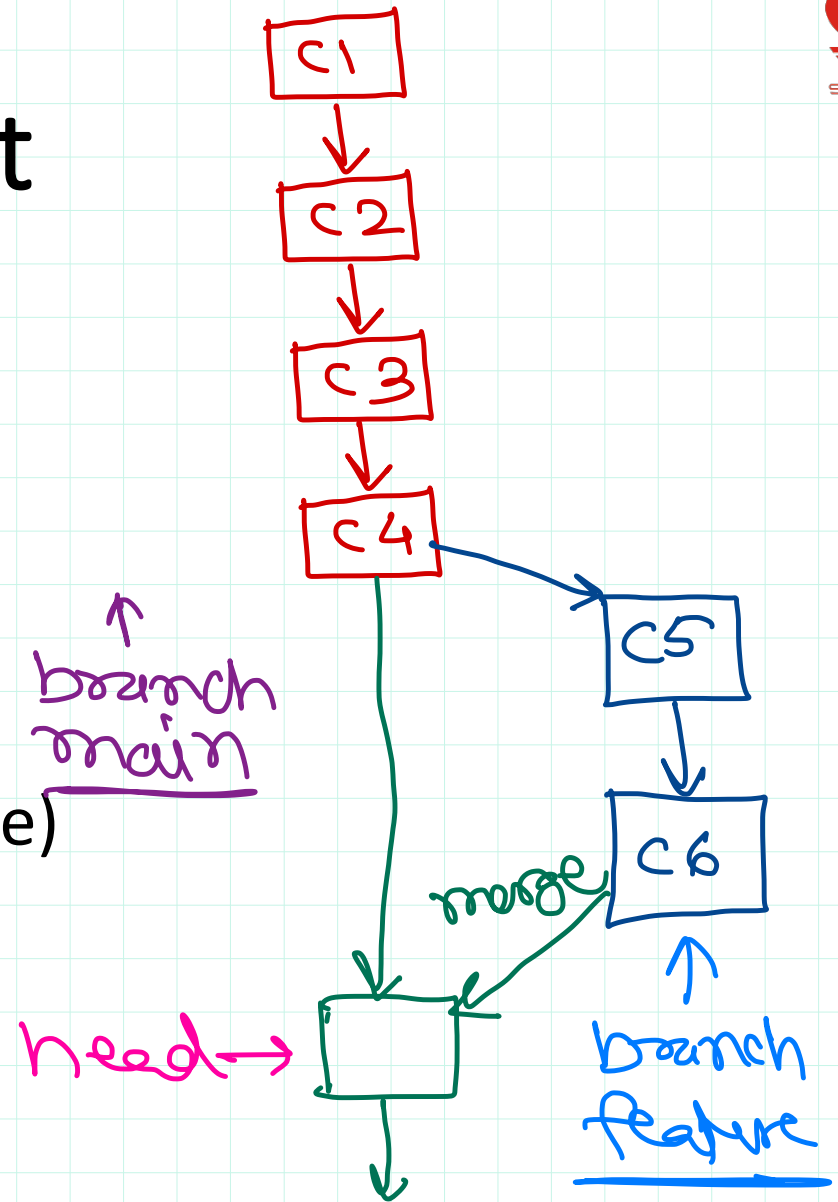


git branch - Branch Management

Purpose: List, create, or delete branches

Syntax: `git branch [options] [<branch-name>]`

- `git branch` - List all local branches
- `git branch -v` - List branches with last commit
- `git branch -a` - List all branches (local and remote)
- `git branch -d <branch>` - Delete merged branch
- `git branch <name>` - Create new branch



git merge - Combining Branches

Purpose: Integrate changes from another branch

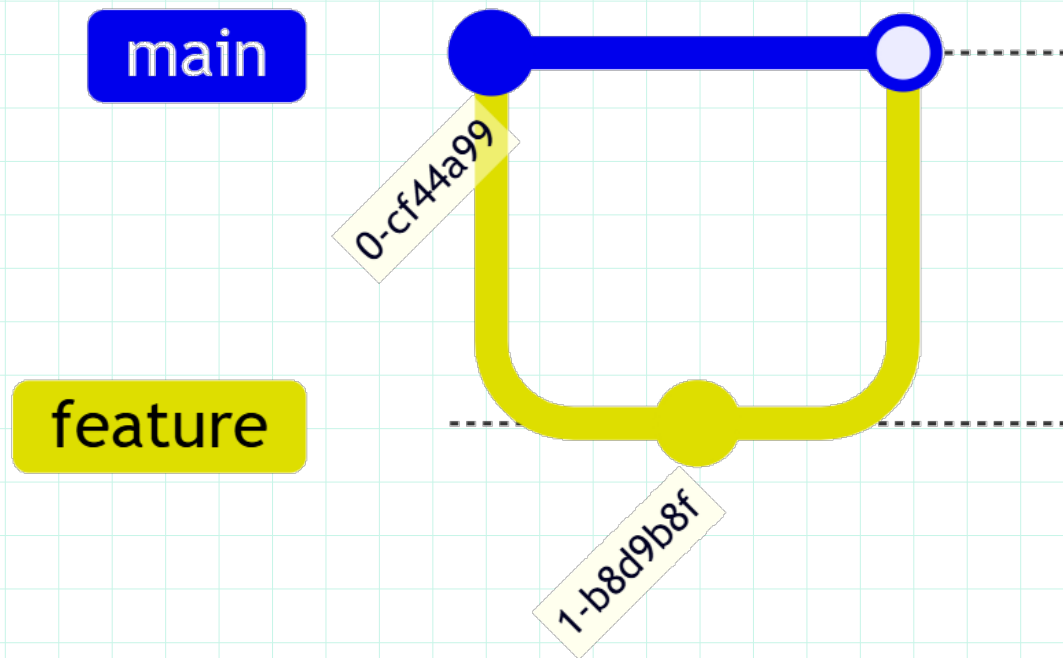
Syntax: `git merge <branch-name>`

Merge Types:

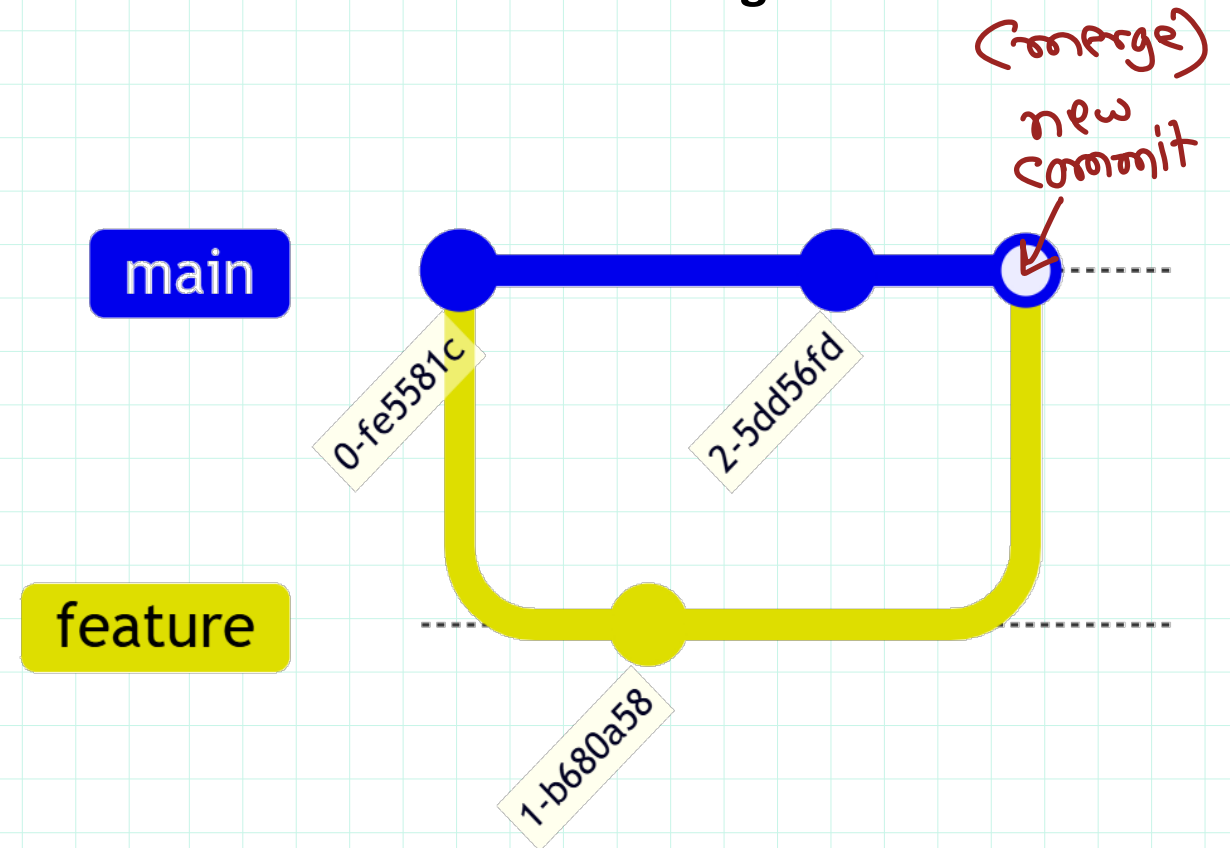
- Fast-Forward Merge: No new commits on target branch → Simply moves branch pointer forward
- 3-Way Merge (Recursive): Both branches have new commits → Creates new merge commit with two parents

Fast Forward vs Recursive Merge

Fast Forward Merge



Recursive Merge



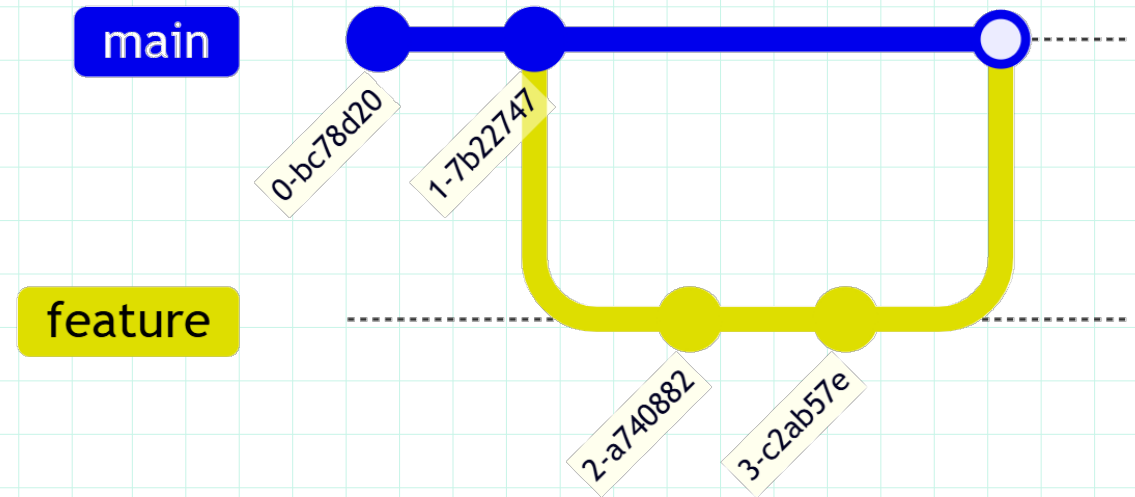
git log - Project History

Purpose: Display commit history

Syntax: `git log [options]`

- `--oneline`: Compact one-line format
- `--graph`: ASCII art branch structure
- `--all`: Show all branches
- `--decorate`: Show branch and tag names

Best Combination: `git log --oneline --graph --all --decorate`



Understanding Merge Conflicts

When Git cannot automatically merge changes - In following situations

- Same lines modified in both branches
- File added in both branches
- File deleted in one branch, modified in another

Conflict Markers:

- <<<<<< HEAD - Your current branch changes
- ===== - Separator between changes
- >>>>>> branch-name - Incoming changes

git rebase - Reapply Commits

Purpose: Re-apply commits on top of another base

Syntax: `git rebase <base-branch>`

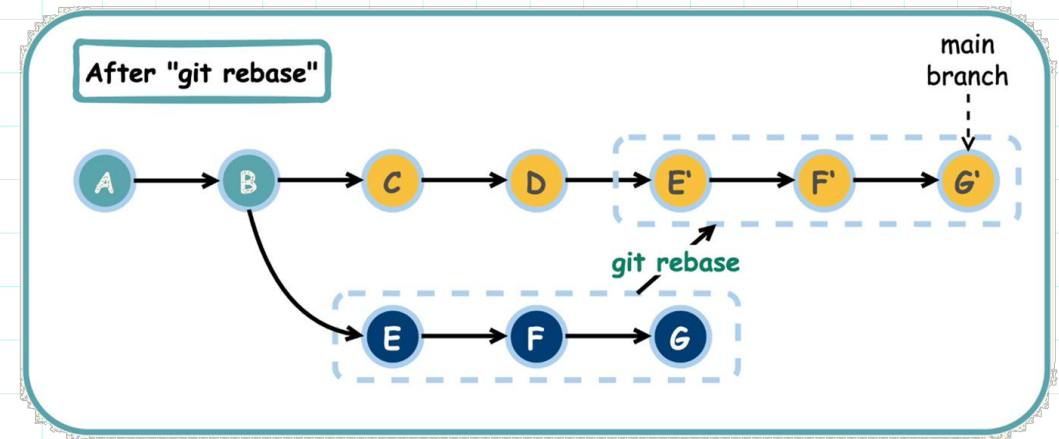
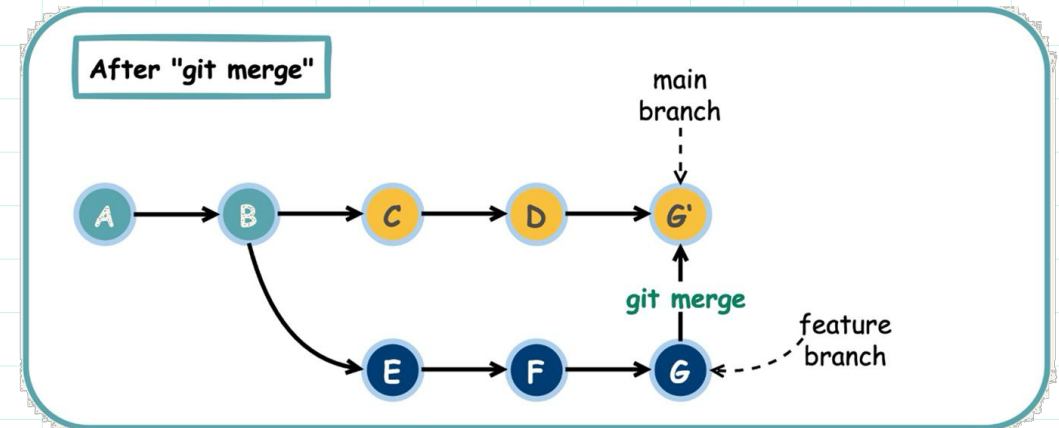
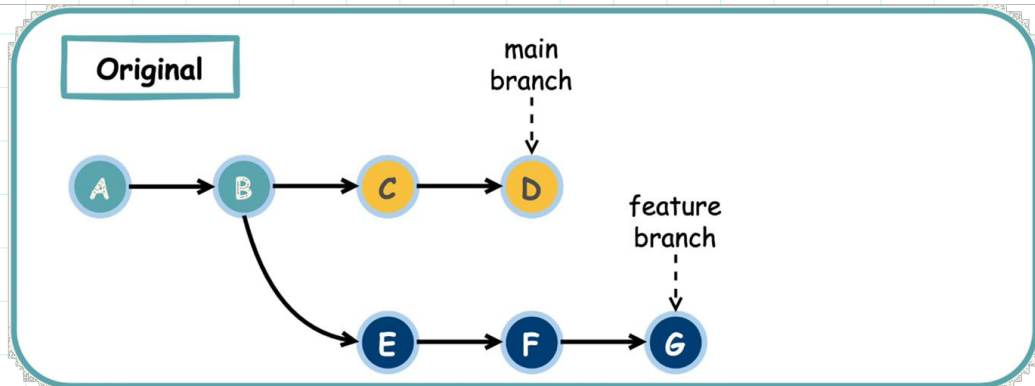
When to Use:

- Clean up commit history before merging
- Update feature branch with latest main
- Create linear history instead of merge commits

⚠ Rebase rewrite commit history - use with caution!

Ⓢ Golden Rule: Ⓢ NEVER rebase commits that have been shared publicly!

Merge vs Rebase

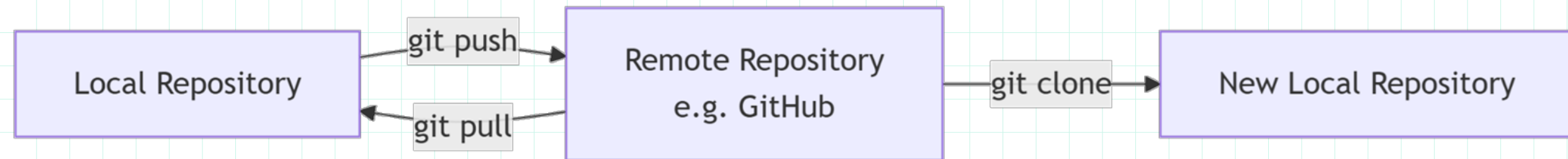


Section 3: Remote Repository Ops

Scenarios: Clone from scratch, Add origin to existing, Pull and push

Learning Objectives:

- Start projects from remote repos
- Connect local repos to remotes
- Synchronize with remote repositories



git clone vs git remote add

Git Clone: Scenario: Start with existing remote repository

- Command: `git clone <url>`
- What it does: Downloads complete repository with history
- Result: Ready-to-work local copy with origin set up

Git Remote Add: Scenario: Connect existing local repo to remote

- Commands: `git remote add origin <url>`
- Follow with: `git push -u origin main`
- Result: Local repo now connected to remote

git pull & git push

- `git push origin main` - Upload local commits to remote
- `git pull origin main` - Download and merge remote changes
- `git fetch origin` - Download without merging (safer option)



Thank You!

Nilesh Ghule

<nilesh@sunbeaminfo.com>